Կոնստրուկտոր և դեստրուկտոր C++ լեզվում

Դասի անդամներ մեթոդներ և կոնստրուկտորների գերբեռնում 80 րոպեանոց դասախոսության նյութ

1 Դասի տվյալների անդամներ և մեթոդներ

Թելադրելու նյութ: Դասի տվյալների անդամները նկարագրում են օբյեկտի վիճակը, իսկ անդամ մեթոդները սահմանում են օբյեկտի հետ թույլատրելի գործողությունները: Յուրաքանչյուր ոչ static օբյեկտ ունի սեփական դաշտերը: Մեթոդը աշխատում է այն օբյեկտի հետ, որի համար կանչվել է:

```
class Student {
private:
   string name;
   int course = 1;

public:
   void setCourse(int value) {
      if (value >= 1 && value <= 4)
         course = value;
   }

   int getCourse() const {
      return course;
   }
};
```

Մեթոդը կարելի է նկարագրել դասի ներսում կամ դասից դուրս՝ ClassName::methodName ձևով: Միայն ընթերցող մեթոդները նշվում են const: Կոնստրուկտորն ու դեստրուկտորը նույնպես դասի հատուկ անդամ մեթոդներ են, սակայն ունեն հատուկ անվանում և կանչի կանոններ:

2 Կոնստրուկտորի գաղափարը

Կոնստրուկտորը հատուկ անդամ մեթոդ է, որը ավտոմատ կանչվում է օբյեկտի ստեղծման պահին: Այն սովորաբար սկզբնարժեքավորում է տվյալների անդամները և ստուգում սկզբնական արժեքները: Կոնստրուկտորի անունը լրիվ համընկնում է դասի անվան հետ և վերադարձվող տիպ չի գրվում, նույնիսկ void:

```
class Student {
private:
   string name;
   int course;

public:
   Student() {
      name = "Unknown";
```

```
    course = 1;
  }
};
```

Student student; հայտարարության ժամանակ C++-ը կանչում է Student() կոնստրուկտորը: Կոնստրուկտորը սովորական մեթոդի պես չի կանչվում student.Student() գրառմամբ:

3 Լռելյայն կոնստրուկտոր

Լռելյայն կոնստրուկտորը այն կոնստրուկտորն է, որը կարելի է կանչել առանց փաստարկի: Այն կարող է ընդհանրապես պարամետր չունենալ կամ բոլոր պարամետրերի համար ունենալ լռելյայն արժեքներ: Եթե ծրագրավորողը ոչ մի կոնստրուկտոր չի հայտարարում, կոմպիլյատորը կարող է ստեղծել անուղղակի լռելյայն կոնստրուկտոր: Եթե հայտարարվում է որևէ այլ կոնստրուկտոր, առանց պարամետրի կառուցումը պետք է անհրաժեշտության դեպքում հայտարարել առանձին:

```
class Rectangle {
private:
  double width;
  double height;

public:
  Rectangle() : width(1), height(1) {
  }
};

Rectangle first;
Rectangle second{};
```

4 Պարամետրերով կոնստրուկտոր

Պարամետրերով կոնստրուկտորը օբյեկտի սկզբնական տվյալները ստանում է ստեղծման արտահայտությունից: Այն թույլ է տալիս օբյեկտը անմիջապես ստեղծել իմաստալից և վավեր վիճակով: Սխալ արժեքների դեպքում կոնստրուկտորը կարող է ընտրել անվտանգ լռելյայն արժեք կամ ավելի ուշ ուսումնասիրվող բացառություն նետել:

```
class Rectangle {
private:
  double width;
  double height;

public:
  Rectangle(double w, double h)
    : width(w > 0 ? w : 1),
      height(h > 0 ? h : 1) {
  }
};

Rectangle room(5.5, 4);
Rectangle screen{16, 9};
```

5 Անդամների սկզբնարժեքավորման ցուցակ

Կոնստրուկտորի վերնագրից հետո գրված երկու կետն սկսում է անդամների սկզբնարժեքավորման ցուցակը: Այստեղ դաշտերը ստեղծվում են իրենց սկզբնական արժեքով, մինչ կոնստրուկտորի մարմնի կատարումը: Այս եղանակը պարտադիր է const տվյալների անդամների, հղումային անդամների և այն անդամ օբյեկտների համար, որոնք չունեն համապատասխան լռելյայն կոնստրուկտոր:

```
class Account {
private:
   const int number;
   string owner;
   double balance;

public:
   Account(int number, const string& owner)
      : number(number),
        owner(owner),
        balance(0) {
   }
};
```

Դաշտերը սկզբնարժեքավորվում են դասում հայտարարման հերթականությամբ, ոչ թե ցուցակում գրված հերթականությամբ: Կարդալու և սխալներից խուսափելու համար ցուցակում ցանկալի է պահպանել դասի դաշտերի հայտարարության հերթը:

6 Կոնստրուկտորների գերբեռնում

Միևնույն դասը կարող է ունենալ մի քանի կոնստրուկտոր, եթե դրանց պարամետրերի ցուցակները տարբերվում են: Սա կոնստրուկտորների գերբեռնումն է: Կոմպիլյատորը օբյեկտի ստեղծման փաստարկների քանակով և տիպերով ընտրում է համապատասխան տարբերակը: Միայն վերադարձվող տիպով գերբեռնում հնարավոր չէ, իսկ կոնստրուկտորն ընդհանրապես վերադարձվող տիպ չունի:

```
class Product {
private:
   string name;
   double price;
   int quantity;

public:
   Product()
      : name("Unknown"), price(0), quantity(0) {
   }

   Product(const string& name, double price)
      : name(name), price(price), quantity(0) {
   }

   Product(const string& name, double price, int quantity)
      : name(name), price(price), quantity(quantity) {
   }
};
```

```
Product first;
Product second("Keyboard", 12000);
Product third("Mouse", 6500, 8);
```

7 Պատվիրակող կոնստրուկտոր

Եթե գերբեռնված կոնստրուկտորները կրկնում են նույն ստուգումները, մեկ կոնստրուկտորը կարող է պատվիրակել աշխատանքը նույն դասի մեկ այլ կոնստրուկտորի: Այդ կոնստրուկտորը նշվում է սկզբնարժեքավորման ցուցակում: Սա նվազեցնում է կրկնվող կողը և պահպանում մեկ ընդհանուր կանոն:

```
class Product {
private:
   string name;
   double price;
   int quantity;

public:
   Product()
      : Product("Unknown", 0, 0) {
   }

   Product(const string& name, double price)
      : Product(name, price, 0) {
   }

   Product(const string& name, double price, int quantity)
      : name(name),
        price(price >= 0 ? price : 0),
        quantity(quantity >= 0 ? quantity : 0) {
   }
};
```

8 Լռելյայն պարամետրեր և անորոշ կանչեր

Լռելյայն պարամետրերը կարող են նվազեցնել կոնստրուկտորների քանակը, բայց սխալ համադրությունը կարող է ստեղծել անորոշ կանչ: Եթե միաժամանակ գոյություն ունի առանց պարամետրի կոնստրուկտոր և բոլոր պարամետրերն ունեցող մեկ այլ կոնստրուկտոր՝ լռելյայն արժեքներով, Product object; գրառման համար երկու տարբերակն էլ հարմար են, և կոմպիլյատորը չի կարող ընտրել:

```
class Product {
public:
   Product();
   Product(string name = "Unknown", double price = 0);
};

// Product item;   // անորոշ կանչ
```

Խնդիրը լուծվում է միջերեսը պարզեցնելով: Պետք է կամ թողնել առանձին առանց պարամետրի կոնստրուկտորը և երկրորդից հեռացնել բոլոր լռելյայն արժեքները, կամ օգտագործել միայն մեկ հստակ կոնստրուկտոր:

9 explicit մեկ պարամետրով կոնստրուկտոր

Մեկ պարամետրով կոնստրուկտորը կարող է աշխատել որպես տիպերի անուղղակի փոխակերպման կանոն: Եթե այդ վարքը ցանկալի չէ, կոնստրուկտորի հայտարարությունից առաջ գրվում է explicit: Այդ դեպքում օբյեկտը պետք է ստեղծել բացահայտ ձևով:

```
class Distance {
private:
   double meters;

public:
   explicit Distance(double meters)
      : meters(meters >= 0 ? meters : 0) {
   }
};

Distance first(15);
Distance second{20};
// Distance third = 25;  // չի թույլատրվում
```

10 Դեստրուկտորի գաղափարը

Դեստրուկտորը հատուկ անդամ մեթոդ է, որը ավտոմատ կանչվում է օբյեկտի կյանքի ավարտին: Նրա անունը սկսվում է ~ նշանով և շարունակվում դասի անունով: Դեստրուկտորը վերադարձվող տիպ և պարամետրեր չունի: Յուրաքանչյուր դաս կարող է ունենալ միայն մեկ դեստրուկտոր, ուստի այն չի գերբեռնվում:

```
class Logger {
private:
   string name;

public:
   Logger(const string& name) : name(name) {
      cout << "Created: " << name << "\n";
   }

   ~Logger() {
      cout << "Destroyed: " << name << "\n";
   }
};
```

11 Օբյեկտի կյանքի տևողությունը

Տեղային օբյեկտը ստեղծվում է կատարման հոսքը հայտարարությանը հասնելու պահին և ոչնչացվում է իր տեսանելիության բլոկից դուրս գալու պահին: Նույն բլոկում ստեղծված օբյեկտները սովորաբար ոչնչացվում են ստեղծման հակառակ հերթականությամբ: Ֆունկցիայի ավարտը, return հրամանը և բացառության պատճառով բլոկից դուրս գալը նույնպես ավարտում են տեղային օբյեկտների կյանքը:

```
int main() {
   Logger first("A");
   {
      Logger second("B");
```

```
    Logger third("C");
  } // նախ C, հետո B
}   // վերջում A
```

12 Օբյեկտների զանգվածի ստեղծում և ոչնչացում

Սովորական օբյեկտների զանգվածի յուրաքանչյուր տարրի համար կոնստրուկտորը կանչվում է ինդեքսների աճման հերթականությամբ: Բլոկի ավարտին դեստրուկտորները կանչվում են հակառակ հերթականությամբ: Առանց սկզբնական արժեքների զանգված ստեղծելու համար դասը պետք է ունենա առանց փաստարկի կանչվող կոնստրուկտոր:

```
Logger items[3] = {
  Logger("A"),
  Logger("B"),
  Logger("C")
};

// ոչնչացման հերթը՝ C, B, A
```

13 Դինամիկ օբյեկտ

new արտահայտությունը ստեղծում է օբյեկտ և կանչում նրա կոնստրուկտորը: delete արտահայտությունը նախ կանչում է դեստրուկտորը, ապա ազատում հիշողությունը: new[]-ով ստեղծված օբյեկտների զանգվածը պետք է ազատել delete[]-ով, որպեսզի բոլոր տարրերի դեստրուկտորները կանչվեն:

```
Logger* one = new Logger("Dynamic");
delete one;
one = nullptr;

Logger* group = new Logger[3];
delete[] group;
group = nullptr;
```

14 Ռեսուրսի կառավարումը դեստրուկտորով

Եթե դասը ձեռք է բերում ռեսուրս, օրինակ՝ new[]-ով հիշողություն, դեստրուկտորը պետք է այն ազատի: Այս մոտեցման շնորհիվ ռեսուրսի կյանքը կապվում է օբյեկտի կյանքի հետ: Ռեսուրսի ստացումը կատարվում է կոնստրուկտորում, իսկ ազատումը՝ դեստրուկտորում:

```
class DynamicArray {
private:
  int* data;
  int size;

public:
  explicit DynamicArray(int size)
    : data(size > 0 ? new int[size]{} : nullptr),
      size(size > 0 ? size : 0) {
  }

  ~DynamicArray() {
```

```
        delete[] data;
    }

    int getSize() const {
        return size;
    }
};
```

Այս պարզ դասի պատճենումը դեռ անվտանգ չէ, քանի որ լռելյայն պատճենումը կպատճենի միայն ցուցիչի հասցեն: Պատճենող կոնստրուկտորն ու վերագրման գործողությունը առանձին թեմա են: Մինչ այդ նման դասի օբյեկտները չպետք է պատճենել կամ կարելի է պատճենումը բացահայտ արգելել:

```
DynamicArray(const DynamicArray&) = delete;
DynamicArray& operator=(const DynamicArray&) = delete;
```

15 Կոնստրուկտորի և դեստրուկտորի համեմատություն

Հատկություն	Կոնստրուկտոր	Սովորական մեթոդ	Դեստրուկտոր
Անուն	դասի անուն	ցանկացած թույլատրելի անուն	~ և դասի անուն
Վերադարձվող տիպ	չունի	ունի կամ void	չունի
Պարամետրեր	կարող է ունենալ	կարող է ունենալ	չունի
Գերբեռնում	կարելի է	կարելի է	չի կարելի
Կանչ	օբյեկտի ստեղծման պահին	ծրագրավորողի կանչով	կյանքի ավարտին

16 Ամբողջական օրինակ Course դասը

```
#include <iostream>
#include <string>
using namespace std;

class Course {
private:
    string title;
    int capacity;
    int enrolled;

public:
    Course()
        : Course("Untitled", 20) {
    }

    Course(const string& title, int capacity)
```

```
      : title(title.empty() ? "Untitled" : title),
        capacity(capacity > 0 ? capacity : 20),
        enrolled(0) {
      cout << "Course created\n";
   }

   Course(const string& title, int capacity, int enrolled)
      : Course(title, capacity) {
      if (enrolled >= 0 && enrolled <= this->capacity)
          this->enrolled = enrolled;
   }

   bool enroll() {
      if (enrolled >= capacity) return false;
      enrolled++;
      return true;
   }

   int freePlaces() const {
      return capacity - enrolled;
   }

   ~Course() {
      cout << "Course destroyed: " << title << "\n";
   }
};
```

```
int main() {
   Course cpp("C++", 25, 20);
   cpp.enroll();
   cout << cpp.freePlaces();
   return 0;
}
```

17 Տարածված սխալներ

1. Կոնստրուկտորի կամ դեստրուկտորի համար վերադարձվող տիպ գրել:

2. Կոնստրուկտորի անունը դասի անունից տարբեր գրել:

3. Դեստրուկտորի անունից առաջ ~ նշանը բաց թողնել:

4. Անդամների սկզբնարժեքավորման ցուցակը երկու կետով չսկսել:

5. Գերբեռնված կոնստրուկտորների համար նույն պարամետրերի ցուցակը գրել:

6. Միաժամանակ ստեղծել երկու առանց փաստարկի հարմար կոնստրուկտոր և ստանալ անորոշ կանչ:

7. new[]-ով ստացված հիշողությունը delete-ով ազատել:

8. Դեստրուկտորը ձեռքով կանչել սովորական տեղային օբյեկտի համար:

9. Ռեսուրս կառավարող դասը պատճենել առանց պատճենման կանոնները սահմանելու:

18 Ամփոփիչ հարցեր

1. Ի՞նչ պահին է ավտոմատ կանչվում կոնստրուկտորը:

2. Ինչո՞վ է կոնստրուկտորը տարբերվում սովորական մեթոդից:

3. Ո՞ր կոնստրուկտորն է կոչվում լռելյայն:

4. Ինչո՞ւ է օգտակար անդամների սկզբնարժեքավորման ցուցակը:

5. Ի՞նչ պայմանով կարելի է գերբեռնել կոնստրուկտորները:

6. Ի՞նչ է պատվիրակող կոնստրուկտորը:

7. Ե՞րբ է նպատակահարմար explicit բանալի բառը:

8. Ե՞րբ է ավտոմատ կանչվում դեստրուկտորը:

9. Ինչո՞ւ դեստրուկտորը չի կարող գերբեռնվել:

10. Ի՞նչ հերթականությամբ են ոչնչացվում նույն բլոկի օբյեկտները:

11. Ինչո՞ւ new[]-ով ստեղծված զանգվածը պետք է ազատել delete[]-ով:

12. Ի՞նչ խնդիր կարող է առաջանալ ռեսուրս կառավարող օբյեկտի լռելյայն պատճենումից: